React Hooks

React Hooks are functions that let you "hook into" React state and lifecycle features in function components. They were introduced in React 16.8, enabling you to use state and other React features without writing a class.

> 💡 **Why Use Hooks?**
>
> 1. **Simplify Code**: Hooks eliminate the need for classes and make your code more readable and maintainable.
> 2. **Reuse Logic**: With custom hooks, you can reuse stateful logic between components.
> 3. **Better Organization**: Hooks separate logic from UI rendering, keeping your components cleaner.

Common React Hooks

1. `useState`: Allows you to add state to a functional component.
2. `useEffect`: Manages side effects in your component.
3. `useContext`: Accesses context values without a wrapper.
4. `useRef`: Creates a reference to a DOM element or value.
5. `useReducer`: An alternative to `useState` for complex state management.
6. `useMemo`: Memoizes a computed value to avoid unnecessary recalculations.
7. `useCallback`: Memoizes a function to prevent its recreation on every render.
8. `useLayoutEffect`: Similar to `useEffect`, but fires synchronously after all DOM mutations.

`useState`

A Hook that lets you add state to functional components.

Definition:

useState is a Hook that returns a stateful value and a function to update it. The initial state is passed to useState, and it preserves the state across re-renders.

Syntax:

```
const [state, setState] = useState(initialState);
```

Code Example:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click Me</button>
    </div>
  );
}

export default Counter;
```

When to Use:

Use `useState` when you need to manage simple state like a counter, form input, or toggling a feature.

useEffect

A Hook that lets you perform side effects in functional components.

Definition:

`useEffect` runs after the render, allowing you to perform operations such as data fetching, subscriptions, or manual DOM manipulations. It optionally cleans up effects to prevent memory leaks.

Syntax:

```
useEffect(() => {
  // Side effect logic here
  return () => {
    // Cleanup (optional)
  };
}, [dependencies]);
```

Code Example:

```
import React, { useState, useEffect } from 'react';

function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setCount(prevCount => prevCount + 1);
    }, 1000);

    return () => clearInterval(interval); // Cleanup
  }, []); // Empty dependency array ensures effect runs once

  return <p>Timer: {count} seconds</p>;
}

export default Timer;
```

When to Use:

Use `useEffect` for:

- Fetching data.
- Subscribing to events.
- Manually modifying the DOM.
- Cleaning up resources (e.g., subscriptions, timers).

useContext

A Hook that lets you access the React context value directly.

Definition:

`useContext` accepts a context object (the value returned from `React.createContext`) and returns the current context value for that context. Context provides a way to pass data through the component tree without having to pass props manually at every level.

Syntax:

```
const value = useContext(MyContext);
```

Code Example:

```
import React, { createContext, useContext } from 'react';

const ThemeContext = createContext('light');
```

```
function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>I am styled by theme context!</button>;
}

export default function App() {
  return (
    <ThemeContext.Provider value="dark">
      <ThemedButton />
    </ThemeContext.Provider>
  );
}
```

When to Use:

Use `useContext` to access global values like themes, authentication status, or settings.

useRef

A Hook that lets you persist values across renders or directly reference a DOM element.

Definition:

`useRef` returns a mutable object whose `.current` property is initialized to the passed argument (`initialValue`). The returned object persists throughout the component's lifecycle.

Syntax:

```
const refObject = useRef(initialValue);
```

Code Example:

```
import React, { useRef } from 'react';

function InputFocus() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus(); // Access the DOM node and set focus
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Click the button to focus me" />
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}

export default InputFocus;
```

When to Use:

- To directly manipulate a DOM element (e.g., focus, scroll).
- To persist values across renders without causing a re-render.

useReducer

A Hook that lets you manage complex state logic in functional components.

Definition:

`useReducer` is an alternative to `useState` for managing more complex state logic. It takes a reducer function and an initial state as arguments and returns the current state along with a dispatch function.

Syntax:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Code Example:

```
import React, { useReducer } from 'react';

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    case 'reset':
      return initialState;
    default:
      throw new Error('Unknown action type');
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
    </div>
  );
}

export default Counter;
```

When to Use:

- For managing complex state logic (e.g., state with multiple sub-values).
- When the next state depends on the previous state.

useMemo

A Hook that lets you optimize performance by memoizing a computed value.

Definition:

`useMemo` accepts a "create" function and a dependency array. It memoizes the result of the "create" function and recomputes it only when one of the dependencies has changed.

Syntax:

```
const cachedValue = useMemo(calculateValue, dependencies)
```

Code Example: Avoiding Expensive Calculations

```jsx
import React, { useState, useMemo } from 'react';

function ExpensiveCalculationExample() {
  const [count, setCount] = useState(0);
  const [input, setInput] = useState('');

  const expensiveCalculation = (num) => {
    console.log('Calculating...');
    return num * 2; // Example of a computational task
  };

  const memoizedValue = useMemo(() => expensiveCalculation(count), [count]);

  return (
    <div>
      <p>Count: {count}</p>
      <p>Result of expensive calculation: {memoizedValue}</p>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
        placeholder="This won't trigger recalculation"
      />
    </div>
  );
}

export default ExpensiveCalculationExample;
```

When to Use:

- For optimizing expensive calculations that depend on specific dependencies.
- When performance improvements are needed for computationally heavy tasks.

useCallback

A Hook that lets you memoize a callback function.

Definition:

`useCallback` returns a memoized version of the callback function that only changes if one of its dependencies has changed. It is useful for preventing unnecessary re-creations of functions.

Syntax:

```jsx
const cachedFn = useCallback(fn, dependencies)
```

Code Example: Preventing Function Recreation

```jsx
import React, { useState, useCallback } from 'react';

function Button({ handleClick }) {
  console.log('Button rendered');
  return <button onClick={handleClick}>Click Me</button>;
}

function ParentComponent() {
  const [count, setCount] = useState(0);

  const memoizedHandleClick = useCallback(() => {
```

```
    console.log('Button clicked!');
  }, []); // No dependencies, so function is memoized

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
      <Button handleClick={memoizedHandleClick} />
    </div>
  );
}

export default ParentComponent;
```

When to Use:

- When passing callback functions to child components to prevent unnecessary re-renders.
- When the same function is used across renders and depends on stable inputs.

useLayoutEffect

A Hook that fires synchronously after all DOM mutations.

Definition:

`useLayoutEffect` is similar to `useEffect`, but it fires synchronously after DOM updates and before the browser paints. It is used for reading layout and synchronously re-rendering.

Syntax:

```
useLayoutEffect(() => {
  // Synchronous effect logic
  return () => {
    // Cleanup (optional)
  };
}, [dependencies]);
```

Code Example: Measuring DOM Layout

```
import React, { useState, useLayoutEffect, useRef } from 'react';

function MeasureWidth() {
  const [width, setWidth] = useState(0);
  const divRef = useRef(null);

  useLayoutEffect(() => {
    setWidth(divRef.current.offsetWidth); // Access DOM width
  }, []);

  return (
    <div>
      <div ref={divRef} style={{ width: '50%' }}>
        Resize me
      </div>
      <p>Width of div: {width}px</p>
    </div>
  );
}

export default MeasureWidth;
```

When to Use:

- When you need to measure or modify DOM elements before the browser paints.
- For tasks requiring synchronous updates to the DOM (e.g., animations).

Summary Table

Hook	Use Case	Best For
`useState`	Manage local state in a functional component	Simple and straightforward state management
`useEffect`	Handle side effects in functional components	Fetching data, subscribing to events, or DOM manipulation
`useContext`	Share data across components without prop drilling	Managing global state or configuration (e.g., theme, user authentication)
`useRef`	Access or persist a DOM element or value without triggering re-renders	DOM manipulation (focus, scroll), persisting mutable values
`useReducer`	Manage complex state logic	Complex states (e.g., form state, counters with multiple actions)
`useMemo`	Memoize expensive computations	Optimizing performance for computationally heavy tasks
`useCallback`	Memoize a function	Preventing function recreation, optimizing child component renders
`useLayoutEffect`	Synchronously handle DOM updates before the browser repaints	Animations, layout measurements, or modifications requiring precision

Notes:

- `useState` is perfect for simpler, component-specific state management.
- `useEffect` handles side effects and can run on mount, unmount, or state updates, making it the most versatile hook.
- `useContext` simplifies state sharing across components, reducing the need for intermediate props.
- `useRef`, `useReducer`, `useMemo`, `useCallback`, and `useLayoutEffect` are generally used in more advanced scenarios where performance or complexity requires specific solutions.